Indian Institute of Technology, Delhi

COL729: Compiler Optimisation

Lab 1: Benchmarking

SPEC CPU2017 Integer Rate Benchmarks
with GCC, Clang and ICC

Prof. Sorav Bansal

Aditya Anand
2023CS50284

Varun Subramaniam
2023CS50497

August 2026

Contents

1 Introduction

This lab investigates the effect of compiler choice, optimization level, and executable bitness on program performance using the SPEC CPU2017 Integer Rate benchmarks.

We evaluate nine benchmarks using GCC, Clang, and ICC, with both 32-bit and 64-bit executables compiled at optimization levels `-O0`, `-O1`, `-O2`, and `-O3`. Execution times and SPEC CPU2017 scores are compared to study how optimization affects different workloads.

We also measure the average number of instructions per basic block for each benchmark.

2 Experimental Setup

We evaluate nine SPEC CPU2017 Integer Rate benchmarks using GCC, Clang, and ICC. Each benchmark was compiled for both 32-bit and 64-bit x86 at optimization levels `-O0`, `-O1`, `-O2`, and `-O3`, giving

$$9 \times 3 \times 2 \times 4 = 216$$

benchmark/configuration measurements.

The benchmarks used are:

Table 1: SPEC CPU2017 Integer Rate benchmarks used in the experiment.

Benchmark	Workload
`500.perlbench_r`	Perl interpreter
`502.gcc_r`	C compiler
`505.mcf_r`	Network-flow optimization
`520.omnetpp_r`	Discrete-event simulation
`523.xalancbmk_r`	XML/XSLT processing
`525.x264_r`	H.264 video encoding
`531.deepsjeng_r`	Chess search
`541.leela_r`	Go game AI
`557.xz_r`	Data compression

Reference inputs were used for all benchmarks with one copy and one iteration (`--size=ref`, `--copies=1`, `--iterations=1`).

For runtime, lower values indicate better performance, while higher SPEC CPU2017 Rate scores indicate better performance. The average number of instructions per basic block was also measured for each configuration, as required by the lab addendum.

2.1 Compiler Flags

For each compiler, the baseline compiler flags were kept fixed while the optimization level was varied from `-O0` to `-O3`.

GCC The GCC configuration used:

```
-g -march=native -fno-unsafe-math-optimizations -fno-tree-loop-vectorize
```

along with the corresponding optimization level.

Clang The Clang configuration used:

```
-mavx
```

along with the corresponding optimization level.

ICC The ICC configuration used `-m32` or `-m64` for the target architecture, along with the corresponding optimization level. No additional optimization flags such as `-xHOST`, IPO, or `-qopt-prefetch` were used.

Thus, for each compiler, the optimization level was varied between `-O0`, `-O1`, `-O2`, and `-O3`, while the remaining configuration was kept fixed.

2.2 Bitness Comparison

The 64-bit speedup over 32-bit was calculated as

$$\text{Speedup}(\%) = \frac{T_{32} - T_{64}}{T_{32}} \times 100,$$

where T_{32} and T_{64} are the corresponding runtimes. A positive value indicates that the 64-bit executable is faster.

2.3 Basic-Block Statistics

For each of the 216 executables, the generated code was disassembled and divided into basic blocks. A new block was started at the program entry point, branch targets, and after control-transfer instructions.

For each executable, we recorded the number of basic blocks N and the total number of instructions. The average number of instructions per basic block was calculated as

$$\bar{I}_{BB} = \frac{\sum_i I_i}{N},$$

where I_i is the number of instructions in basic block i.

3 Overall Runtime Trends

Figure 1 summarizes the runtime of the nine benchmarks across the different optimization levels. The most noticeable trend is the sharp decrease in runtime between $-O0$ and $-O1$ across nearly all benchmarks and compilers. Further improvements are smaller, and the curves often flatten or increase at $-O3$.

Table 2: Average runtime change produced by successive optimization levels.

Transition	Mean reduction	Median reduction	Interpretation
$-O0 \rightarrow -O1$	57.3%	53.2%	Very large and consistently beneficial
$-O1 \rightarrow -O2$	11.4%	8.9%	Significant, but workload dependent
$-O2 \rightarrow -O3$	−0.2%	0.3%	Essentially neutral on average

The transition from $-O0$ to $-O1$ has the largest effect, with a mean runtime reduction of 57.3%. Moving to $-O2$ provides a further 11.4% reduction on average, giving an overall reduction of about

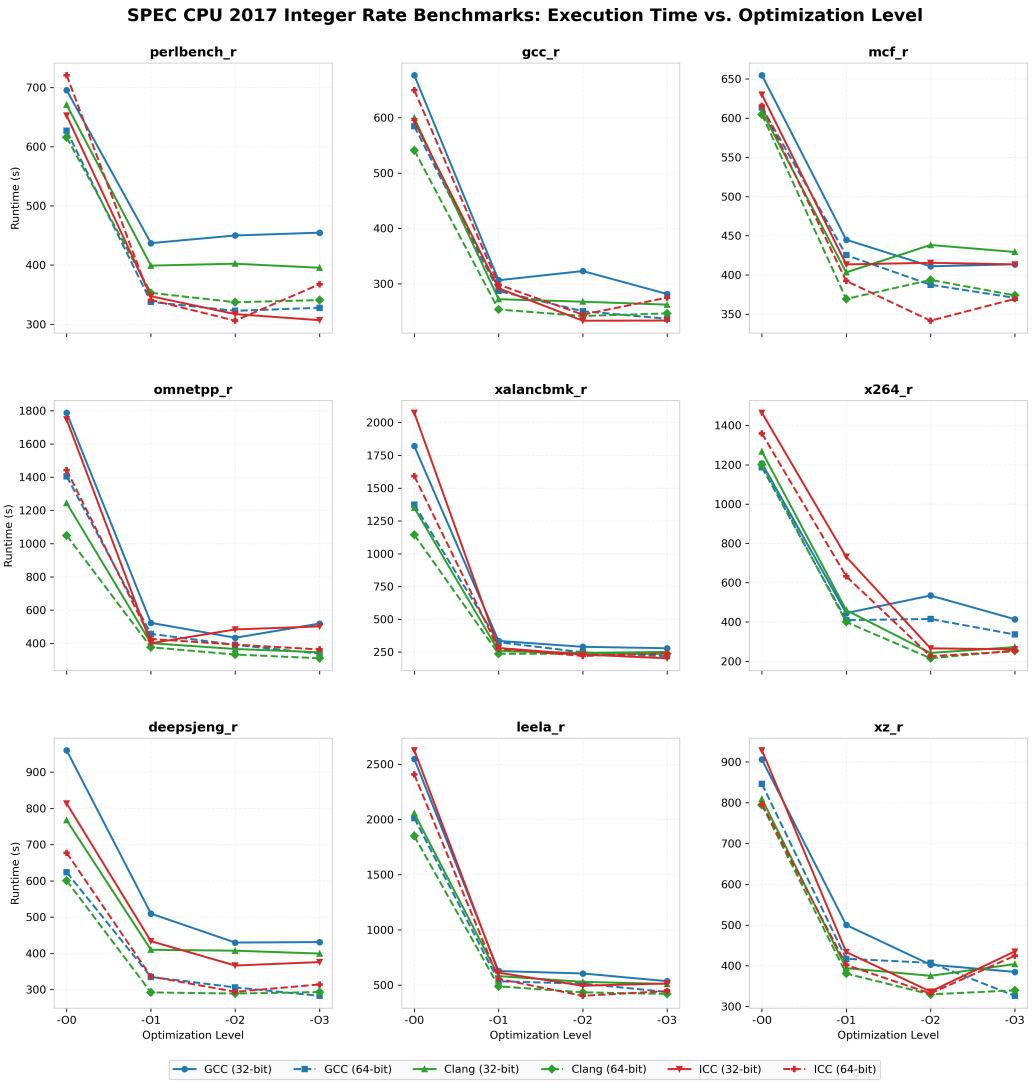

Figure 1: Execution time versus optimization level for the nine SPEC CPU2017 Integer Rate benchmarks. Solid lines denote 32-bit executables and dashed lines denote 64-bit executables; colors identify GCC, Clang, and ICC.

61.9% relative to $-O0$. In contrast, $-O3$ provides almost no additional improvement over $-O2$, with a mean change of -0.2%.

The results also show that $-O3$ is not consistently better than $-O2$. It is faster in 28 of the 54 series and slower in 26. Thus, the benefit of higher optimization levels depends on both the compiler and the benchmark.

4 Question 1: Which Optimizations Are Most and Least Consequential?

The effect of increasing the optimization level is summarized in Table 3.

Table 3: Aggregate effect of increasing optimization level.

Transition	Mean runtime reduction	Cases improved
$-O0 \rightarrow -O1$	57.3%	54/54
$-O1 \rightarrow -O2$	11.4%	45/54
$-O2 \rightarrow -O3$	-0.2%	28/54

The **most consequential** transition is $-O0 \rightarrow -O1$, which reduces runtime by 57.3% on average and improves all 54 benchmark. The large improvement is consistent with the substantial optimizations introduced at this stage, such as dead-code elimination, propagation and simplification, improved register allocation, and basic inlining. The effect also varies considerably across benchmarks: the average reduction is approximately 81.2% for `523.xalancbmk_r`, 74.6% for `541.leela_r`, and 69.6% for `520.omnetpp_r`, while `505.mcf_r` is considerably less sensitive to optimization.

The transition $-O1 \rightarrow -O2$ is the **second most consequential**, with a further average runtime reduction of 11.4%. However, only 45 of the 54 series improve (9 regress), showing that the benefit of additional optimization is already becoming more workload dependent. For example, `525.x264_r` shows a substantial benefit from moving to $-O2$.

The **least consequential** transition is $-O2 \rightarrow -O3$. Its average effect is only -0.2%, with 28 of 54 series becoming faster and 26 becoming slower. This is consistent with the fact that many important optimizations have already been applied at lower levels, so the additional transformations at $-O3$ have more workload- and compiler-dependent effects. More aggressive transformations can improve some workloads, but may also increase code size, register pressure, or instruction-cache pressure.

Overall, $-O0 \rightarrow -O1$ is the **most consequential** transition, while $-O2 \rightarrow -O3$ is the **least consequential**. The results show that the benefit of optimization decreases substantially after $-O1$ and becomes increasingly dependent on the benchmark and compiler.

5 Question 2: Which Compilers Are Better, and in What Aspects?

No single compiler is consistently fastest across all benchmarks. Looking at the single fastest configuration for each benchmark (Table 7, presented in full in Section 8), ICC produces the best overall result for five of the nine benchmarks, while Clang and GCC each produce the best result for two. However, this "best single configuration" view only tells part of the story, since it collapses 24 measurements per benchmark down to a single winner. A finer-grained view, tallying which compiler is fastest in *every* individual (bitness, optimization level) cell, reveals a more nuanced picture.

5.1 Cell-wise win counts

For each of the 9 benchmarks × 2 bitness choices × 4 optimization levels = 72 configurations, we identified which of the three compilers produced the lowest runtime. Table 4 breaks this down by optimization level, and Table 5 breaks it down by benchmark.

Table 4: Number of lowest-runtime compiler wins across the 18 benchmark/bitness cases at each optimization level.

Optimization	Clang	GCC	ICC
$-O0$	14	2	2
$-O1$	14	2	2
$-O2$	7	1	10
$-O3$	4	6	8
Total	**39**	**11**	**22**

Table 5: Number of lowest-runtime compiler wins across the eight bitness/optimization configurations for each benchmark.

Benchmark	Clang	GCC	ICC
`500.perlbench_r`	1	2	5
`502.gcc_r`	4	1	3
`505.mcf_r`	4	1	3
`520.omnetpp_r`	7	0	1
`523.xalancbmk_r`	4	1	3
`525.x264_r`	3	3	2
`531.deepsjeng_r`	5	1	2
`541.leela_r`	6	0	2
`557.xz_r`	5	2	1
Total	**39**	**11**	**22**

This cell-wise count is not in conflict with the "best single configuration" result above; it simply views the data at finer granularity. Clang actually wins the plurality of individual cells (39/72, or 54%), because it is overwhelmingly dominant at low optimization levels: it wins 14 of 18 cases at both $-O0$ and $-O1$. ICC's strength, on the other hand, is concentrated almost entirely at $-O2$ and $-O3$, where it wins 18 of its 22 total cells. Since the single fastest configuration for most benchmarks happens to occur at $-O2$ or $-O3$ (because runtime is lowest there), ICC ends up disproportionately represented among the "best overall" picks even though it does not win the majority of individual comparisons.

5.2 ICC: strongest at high optimization levels

Under the tested configurations, ICC produces the fastest overall configuration for five of nine benchmarks:

- `500.perlbench_r`: 64-bit, $-O2$, 306.0 s;
- `502.gcc_r`: 32-bit, $-O2$, 233.4 s;
- `505.mcf_r`: 64-bit, $-O2$, 341.6 s;
- `523.xalancbmk_r`: 32-bit, $-O3$, 202.7 s;

- `541.leela_r`: 64-bit, $-O2$, 402.2 s.

ICC is particularly effective at $-O2$: for eight of the nine benchmarks, $-O2$ gives the lowest runtime among the four optimization levels for the 64-bit executable — the sole exception is `520.omnetpp_r`, where $-O3$ (362.6 s) edges out $-O2$ (391.8 s). This suggests that, for the workloads tested, ICC often reaches a favorable balance between optimization and code generation at $-O2$, while the additional transformations at $-O3$ do not consistently improve performance. This is also visible in Table 4: ICC accounts for 10 of the 18 $-O2$ wins and 8 of the 18 $-O3$ wins, but almost none of the $-O0$/$-O1$ wins.

5.3 Clang: dominant at low optimization, strong on specific workloads

Table 4 shows that Clang wins 14 of 18 cases at both $-O0$ and $-O1$ — by far the largest share of any compiler at those levels. Clang also produces the fastest overall configurations for `520.omnetpp_r` and `525.x264_r`. The best result for OMNeT++ is the 64-bit $-O3$ configuration at 309.6 s, while x264 achieves its best result with 64-bit $-O2$ at 215.3 s. Table 5 shows Clang is particularly dominant on `520.omnetpp_r` (7/8 wins) and `541.leela_r` (6/8 wins).

Clang also shows the most consistent 64-bit advantage: the 64-bit executable is faster in all 36 of the paired 32-bit/64-bit comparisons.

5.4 GCC: strongest 64-bit advantage, competitive at $-O3$

GCC produces the fastest overall configurations for `531.deepsjeng_r` and `557.xz_r`. Its most notable characteristic is the size and consistency of its 64-bit advantage. The mean speedup of 64-bit over 32-bit increases from 15.6% at $-O0$ to 21.9% at $-O3$, and the 64-bit executable is faster in 35 of the 36 paired comparisons. GCC's cell-wise wins are concentrated at $-O3$ (6/18), where it overtakes Clang's declining share (Table 4).

Table 6: Mean 64-bit speedup over 32-bit by compiler and optimization level.

Compiler	$-O0$	$-O1$	$-O2$	$-O3$
GCC	15.6%	13.7%	16.2%	21.9%
Clang	9.8%	11.4%	13.1%	12.9%
ICC	7.7%	5.8%	10.6%	2.1%

Figure 2 shows the corresponding 64-bit speedup across optimization levels.

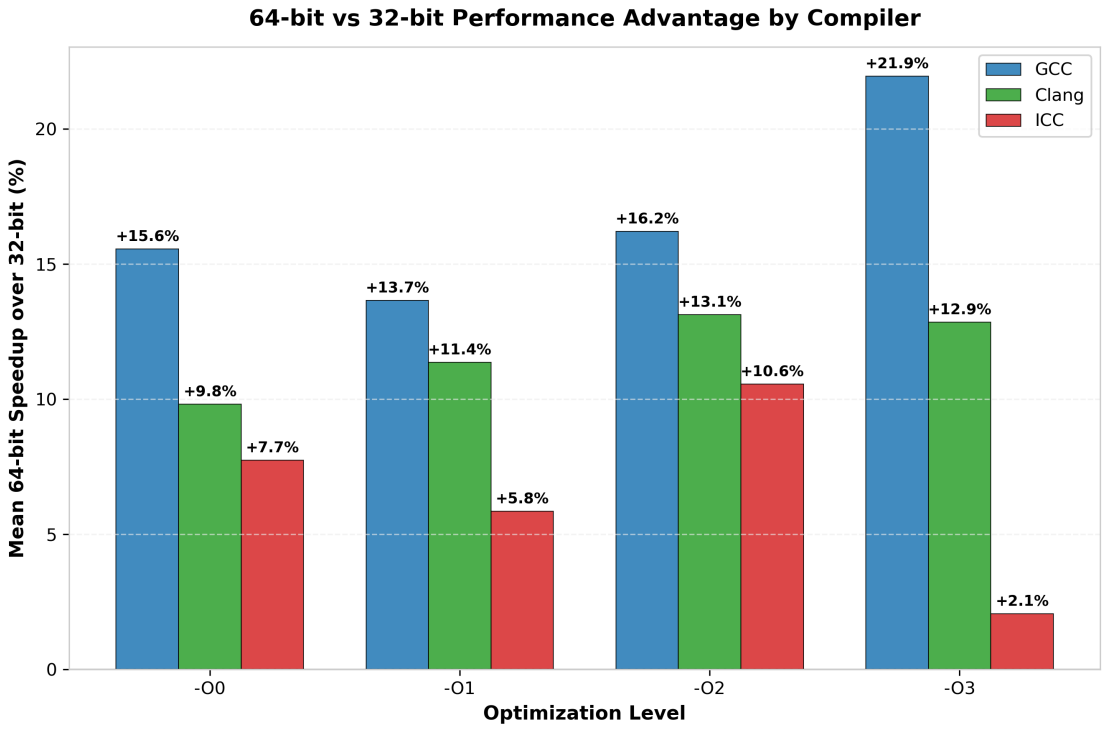

Figure 2: Mean 64-bit performance advantage over 32-bit for each compiler and optimization level.

Overall, ICC's overall best-case results are concentrated at $-O2/-O3$, while Clang wins the largest share of individual configurations, driven by its dominance at $-O0/-O1$. GCC shows the largest and most consistent benefit from 64-bit execution. These results indicate that compiler choice should depend on both the workload and the intended optimization level, rather than assuming that one compiler is universally better.

6 Question 3: Which Are Faster, 32-bit or 64-bit Executables? Why?

The overall answer is clear: **64-bit executables are usually faster**, although the advantage is not universal for every compiler and configuration. Across the entire dataset, Clang's 64-bit version wins all 36 paired comparisons, GCC wins 35 of 36, and ICC wins 28 of 36.

The data show several plausible architectural reasons for this result.

First, x86-64 provides a substantially richer register set than the legacy 32-bit x86 environment. More registers reduce the need to spill values to memory, which can directly reduce instruction count and memory traffic. This can be especially useful for workloads with many simultaneous live scalar values.

Second, the x86-64 ABI and instruction set provide cleaner support for modern compiler-generated code. Compilers can pass more arguments in registers and can exploit a 64-bit instruction set that was designed alongside modern microarchitectural features.

Third, many 64-bit builds are more likely to expose the compiler's full target-specific optimization capabilities. Even when the source program does not require 64-bit integers, the surrounding calling convention, register allocation, addressing modes, and generated instruction sequences can still produce better machine code.

However, 64-bit code is not automatically faster. It can increase pointer size and sometimes increase memory footprint, which can hurt cache locality. The supplied ICC data illustrate this clearly: ICC's 32-bit executable beats its 64-bit executable in all eight of the (8/36) cases where 32-bit wins overall.

This is most striking for `502.gcc_r`, where ICC's 32-bit build is faster than its 64-bit build at *every* optimization level ($-O0$ through $-O3$); 32-bit also wins for `500.perlbench_r` at $-O0$ and $-O3$, for `520.omnetpp_r` at $-O1$, and for `523.xalancbmk_r` at $-O3$. This shows that the answer is an empirical one: 64-bit usually wins because of the execution model and available resources, but memory-system behavior and compiler code generation can reverse the outcome for specific benchmarks.

7 Question 4: How Do the Optimization Levels Differ Across Compilers?

The optimization levels represent increasingly aggressive compiler optimization, but their exact behavior is compiler-specific. In general, $-O0$ performs little or no optimization, $-O1$ enables basic performance-oriented transformations, $-O2$ enables a broader set of optimizations, and $-O3$ allows still more aggressive transformations. However, these levels are not standardized bundles: GCC, Clang, and ICC choose different optimization passes, heuristics, and transformations at each level.

7.1 $-O0$: Unoptimized baseline

At $-O0$, all three compilers produce substantially slower executables. This provides the baseline against which the effect of optimization can be measured. The differences between compilers at this level also show that compiler choice matters even before more aggressive optimizations are enabled.

7.2 $-O1$: Largest improvement

The transition from $-O0$ to $-O1$ produces the largest performance improvement in the experiment. Across all benchmark/compiler/bitness series, it reduces runtime by an average of 57.3%, with all 54 series improving. This indicates that a large fraction of the performance benefit comes from moving from unoptimized to optimized code.

7.3 $-O2$: Further significant improvement

Moving from $-O1$ to $-O2$ provides a further average runtime reduction of 11.4%. The improvement is smaller than that of $-O0 \rightarrow -O1$ and is more dependent on the compiler and benchmark. ICC benefits particularly strongly from this level in the tested configurations, with $-O2$ giving the best result for most of its benchmark/bitness combinations.

7.4 $-O3$: More aggressive, but not consistently faster

The additional optimization at $-O3$ does not provide a consistent improvement over $-O2$. Across the 54 series, the average change is -0.2%, with 28 series becoming faster and 26 becoming slower. Thus, although $-O3$ can improve particular workloads, its additional transformations can also result in little improvement or a small regression.

The behavior therefore differs across compilers. ICC gains substantially when moving from $-O1$ to $-O2$ and generally sees little additional benefit at $-O3$, while GCC and Clang show different responses depending on the benchmark. This demonstrates that the same optimization label does not correspond to exactly the same set of transformations across compilers.

Overall, the results show that optimization levels should not be viewed as universal guarantees of performance. The largest improvement occurs at $-O0 \rightarrow -O1$, while the benefit of moving beyond $-O2$ is much more dependent on the compiler and workload.

8 Question 5: Does the Kind of Benchmark Influence the Compiler Comparison?

Yes. The compiler producing the fastest configuration changes across benchmarks. ICC gives the fastest configuration for five of the nine benchmarks, while Clang and GCC each win two (Table 7). This shows that there is no single compiler that consistently performs best across all workloads.

Table 7: Fastest measured configuration for each benchmark across all compilers, bitness choices, and optimization levels.

Benchmark	Compiler	Bitness	Opt.	Runtime (s)
`500.perlbench_r`	ICC	64-bit	$-O2$	305.95
`502.gcc_r`	ICC	32-bit	$-O2$	233.39
`505.mcf_r`	ICC	64-bit	$-O2$	341.62
`520.omnetpp_r`	Clang	64-bit	$-O3$	309.62
`523.xalancbmk_r`	ICC	32-bit	$-O3$	202.72
`525.x264_r`	Clang	64-bit	$-O2$	215.29
`531.deepsjeng_r`	GCC	64-bit	$-O3$	282.61
`541.leela_r`	ICC	64-bit	$-O2$	402.24
`557.xz_r`	GCC	64-bit	$-O3$	326.40

The reason is that different benchmarks place different demands on the compiler and processor. Compiler optimizations that benefit one type of workload may provide little benefit, or even hurt performance, on another.

For example, `505.mcf_r` is a graph and optimization workload, while `525.x264_r` is a multimedia encoding workload. Their best-performing compilers are different: ICC is fastest for MCF, whereas Clang is fastest for x264. Similarly, GCC performs best for the game-search benchmark `531.deepsjeng_r` and the compression benchmark `557.xz_r`, while ICC performs best for `500.perlbench_r`, `502.gcc_r`, and `541.leela_r`.

The benchmark also affects which optimization level performs best. For example, the best configuration for `523.xalancbmk_r` uses ICC at $-O3$, whereas both `525.x264_r` and `541.leela_r` achieve their best results with $-O2$. Thus, compiler choice and optimization level should be considered together rather than independently of the workload.

Overall, the results confirm that **the type of benchmark has a significant influence on compiler performance**. A compiler that performs well for one workload is not necessarily the best choice for another. The benchmark mix therefore matters. A compiler that performs particularly well on pointer-heavy C++ workloads need not be the best choice for video coding, and a compiler that excels on branch-heavy search kernels may not dominate on XML transformation code. The full runtime and SPECrate curves for each individual benchmark are provided in Appendix A for reference.

9 Why the Results Look This Way

The observed performance differences can be interpreted using the interaction between compiler transformations, instruction-level execution, and memory behavior.

9.1 Instruction count is only part of the story

A lower instruction count often helps, but it is not sufficient to guarantee a lower runtime. Reordering instructions can improve pipeline utilization without reducing total instructions. Conversely, loop unrolling may increase instruction count while reducing branch overhead and increasing available instruction-level parallelism. Therefore, a compiler can produce a larger binary and still execute it

faster. This is visible directly in our data: static instruction counts and average basic-block sizes both *rise slightly* from $-O1$ to $-O3$ for most compilers (Table 10) even though runtime keeps falling (or stays flat) over the same range — the extra instructions from inlining and unrolling are more than compensated for by reduced branching and better scheduling.

9.2 Code size and cache effects

Aggressive inlining and loop transformations can increase code size. When that pushes hot code beyond instruction-cache capacity, performance can degrade. This is a plausible explanation for several of the observed $-O3$ regressions, especially where $-O2$ is clearly better. The basic-block data support this mechanism: for ICC, static instruction counts and block sizes continue to climb from $-O2$ to $-O3$ on several benchmarks even as measured runtime stagnates or worsens, consistent with diminishing returns from additional code growth.

9.3 Register allocation and 64-bit code

The larger register set in the x86-64 execution model is one reason the 64-bit binaries often win. Fewer spills and more register-resident values can be especially beneficial for loops with many live variables. However, larger pointers and data structures can increase memory traffic and reduce cache density, explaining why a 32-bit result can occasionally be faster.

9.4 Compiler cost models

GCC, Clang, and ICC use different optimization passes, profitability heuristics, target-specific decisions, and internal cost models. Consequently, an optimization such as inlining, loop unrolling, or vectorization may be profitable for one compiler but not another on exactly the same source program. The experimental data are therefore an empirical demonstration of compiler "personality," and this personality is visible not just in runtime but in the static shape of the generated code, as Table 10 shows: GCC's average block size drops sharply from $-O0$ (6.93) down to $-O1$ (5.09) and then only creeps back up to 5.38 by $-O3$, whereas ICC starts from the largest unoptimized blocks of the three compilers (7.35) and settles at a comparable 5.60 by $-O3$ — the three compilers converge toward broadly similar optimized block sizes despite very different starting points and very different runtime outcomes.

10 Basic-Block Instruction Count Analysis

This section examines the average number of instructions per basic block for each benchmark. Shorter blocks mean more frequent branches; longer blocks give the processor more sequential instructions to schedule.

10.1 Methodology and Assumptions

The basic-block analysis is performed statically on the final x86 ELF executables generated for each benchmark, compiler, bitness, and optimization level. The compiled binaries are disassembled using `objdump`, and the resulting instruction stream is used to identify basic blocks.

A basic block is treated as a straight-line sequence of instructions that ends at a control-transfer instruction such as a jump, call, return, loop, or system call. The instruction following a control-transfer instruction starts a new block, and function boundaries are also treated as block boundaries.

The analysis is applied independently to every compiled configuration so that differences caused by the compiler, target bitness, or optimization level are reflected in the results. The reported value is

the average number of static instructions per basic block. It is not weighted by runtime execution frequency.

10.2 Averages by benchmark, compiler, and bitness

Table 8 reports the average instructions per basic block for each benchmark. The representative build uses GCC, 64-bit, $-O2$, while the all-configs values summarize the 24 compiler/bitness/optimization combinations for each benchmark.

Table 8: Average instructions per basic block, per benchmark. "Representative build" uses GCC, 64-bit, $-O2$. "All-configs" summarizes over all 24 compiler/bitness/optimization combinations for that benchmark.

Benchmark	Representative build (GCC 64-bit $-O2$)	All-configs (24 builds) Mean	Min	Max
500.perlbench_r	4.12	4.88	3.88	6.58
502.gcc_r	3.65	4.36	3.43	5.97
505.mcf_r	5.33	6.98	4.71	10.37
520.omnetpp_r	4.05	4.68	3.88	5.73
523.xalancbmk_r	4.16	4.78	3.93	5.79
525.x264_r	6.53	7.77	6.15	10.13
531.deepsjeng_r	5.01	6.24	4.59	9.35
541.leela_r	4.54	5.31	4.17	6.44
557.xz_r	5.10	6.70	5.10	8.84

The largest average block size is observed for `525.x264_r`, at 7.77 instructions per block, while `502.gcc_r` has the smallest, at 4.36. This shows that the structure of the generated code varies substantially across benchmarks.

Table 9 shows the same statistic grouped by compiler and bitness. For all three compilers, the 64-bit configurations have smaller average blocks than the corresponding 32-bit configurations. The difference is largest for Clang, whose average decreases from 6.54 to 4.93 instructions per block.

Table 9: Average instructions per basic block by compiler and bitness (36 measurements per row).

Compiler	Bitness	Mean	Min	Max
GCC	32-bit	6.10	4.18	10.37
GCC	64-bit	5.21	3.43	9.60
Clang	32-bit	6.54	4.87	9.11
Clang	64-bit	4.93	3.79	7.20
ICC	32-bit	5.94	4.04	10.10
ICC	64-bit	5.75	3.70	10.13

10.3 Trend across optimization levels

Table 10 shows the mean block size for each compiler and optimization level. Across all compilers, the average decreases from 6.69 instructions per block at $-O0$ to 5.17 at $-O1$, and then increases slightly to 5.52 and 5.59 at $-O2$ and $-O3$, respectively.

The trend differs across compilers. GCC and ICC show a larger decrease from $-O0$ to $-O1$, while Clang remains relatively stable across the four optimization levels. Thus, increasing the optimization level changes the structure of the generated code, but the effect on basic-block size is not uniform across compilers.

Table 10: Mean instructions per basic block by compiler and optimization level (18 builds per cell).

Compiler	$-O0$	$-O1$	$-O2$	$-O3$
GCC	6.93	5.09	5.20	5.38
Clang	5.79	5.55	5.79	5.81
ICC	7.35	4.86	5.57	5.60
All compilers	**6.69**	**5.17**	**5.52**	**5.59**

Basic-block size does not directly determine runtime. For example, `505.mcf_r` has a relatively large average block size but is among the less optimization-sensitive benchmarks, whereas `523.xalancbmk_r` has a smaller average block size but is highly sensitive to optimization. Therefore, the average block size is best viewed as a structural characteristic of the generated code rather than as a direct performance predictor.

10.4 Full per-optimization-level results

Tables 11–14 give the complete average-instructions-per-basic-block figures for all six compiler/bitness combinations at each optimization level.

Table 11: Average static instructions per basic block at `-O0`.

Benchmark	GCC-32	GCC-64	Clang-32	Clang-64	ICC-32	ICC-64
`500.perlbench_r`	6.545	6.241	5.835	4.997	6.583	6.559
`502.gcc_r`	5.154	4.764	5.324	4.091	5.965	5.709
`505.mcf_r`	10.366	9.067	8.023	6.414	10.100	8.448
`520.omnetpp_r`	5.376	4.970	5.534	3.880	5.730	5.726
`523.xalancbmk_r`	5.380	5.219	5.578	3.932	5.785	5.719
`525.x264_r`	9.146	9.597	8.298	7.198	9.718	10.133
`531.deepsjeng_r`	9.350	7.327	6.761	5.276	9.222	8.589
`541.leela_r`	5.587	5.291	6.000	4.172	6.210	5.643
`557.xz_r`	7.806	7.602	7.184	5.793	8.491	8.037

Table 12: Average static instructions per basic block at `-O1`.

Benchmark	GCC-32	GCC-64	Clang-32	Clang-64	ICC-32	ICC-64
`500.perlbench_r`	4.531	3.884	5.236	4.209	4.305	4.059
`502.gcc_r`	4.181	3.433	4.873	3.788	4.037	3.700
`505.mcf_r`	7.140	5.221	8.488	5.050	5.704	4.708
`520.omnetpp_r`	4.940	3.888	5.080	3.902	4.813	4.091
`523.xalancbmk_r`	4.852	3.956	5.432	3.957	4.600	3.999
`525.x264_r`	7.049	6.292	8.115	6.326	6.485	6.152
`531.deepsjeng_r`	6.274	4.863	7.084	5.053	5.322	4.588
`541.leela_r`	5.275	4.320	5.998	4.409	5.244	4.560
`557.xz_r`	6.214	5.314	7.332	5.508	5.902	5.208

Table 13: Average static instructions per basic block at `-O2`.

Benchmark	GCC-32	GCC-64	Clang-32	Clang-64	ICC-32	ICC-64
`500.perlbench_r`	4.799	4.118	5.310	4.300	4.299	4.153
`502.gcc_r`	4.401	3.649	4.950	3.902	4.039	3.755
`505.mcf_r`	7.179	5.327	9.095	5.969	6.343	5.152
`520.omnetpp_r`	5.053	4.048	5.117	3.955	4.750	4.273
`523.xalancbmk_r`	5.104	4.159	5.555	4.091	4.803	4.222
`525.x264_r`	7.361	6.533	8.558	6.821	7.529	8.000
`531.deepsjeng_r`	5.908	5.011	7.510	5.447	5.286	5.787
`541.leela_r`	5.316	4.541	5.846	4.531	5.763	6.440
`557.xz_r`	5.957	5.104	7.453	5.731	6.892	8.836

Table 14: Average static instructions per basic block at `-O3`.

Benchmark	GCC-32	GCC-64	Clang-32	Clang-64	ICC-32	ICC-64
`500.perlbench_r`	4.839	4.157	5.296	4.297	4.329	4.195
`502.gcc_r`	4.449	3.694	4.947	3.908	4.062	3.783
`505.mcf_r`	7.499	5.430	9.109	5.923	6.501	5.174
`520.omnetpp_r`	5.042	3.999	5.118	3.946	4.749	4.267
`523.xalancbmk_r`	5.305	4.297	5.556	4.093	4.826	4.246
`525.x264_r`	8.300	7.451	8.799	7.017	7.597	8.064
`531.deepsjeng_r`	5.937	4.949	7.679	5.441	5.293	5.798
`541.leela_r`	5.508	4.513	5.726	4.471	5.776	6.379
`557.xz_r`	6.286	5.228	7.492	5.690	6.920	8.757

11 Resources

The complete results, plots, and supporting files for this experiment are available at:

https://github.com/AdityaCodeSphere123/SPEC-CPU2017-Benchmarking

The repository contains the benchmark results and plots used in this report.

12 References

1. SPEC CPU2017 Documentation https://www.spec.org/cpu2017/Docs/overview.html

2. COL729 Lab 1 assignment handout.

A Per-Benchmark Runtime and SPECrate Plots

For completeness, this appendix presents the detailed runtime and SPECrate curves for each of the nine benchmarks individually (Figures 3–11). Each figure shows execution time (top) and SPECrate2017_int base score (bottom) versus optimization level, for all six compiler/bitness combinations. These are the same underlying measurements summarized in Figure 1 and discussed benchmark-by-benchmark in Section 8; they are included here in full for reference.

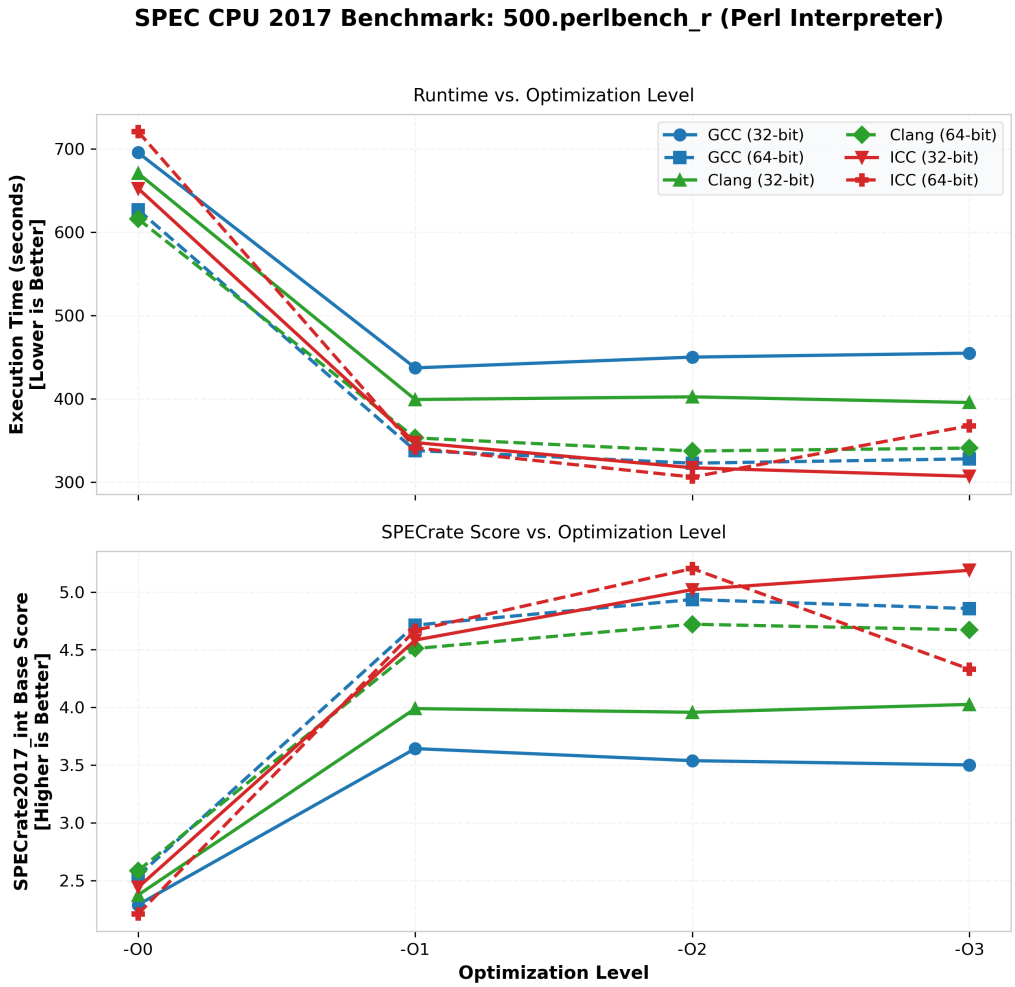

Figure 3: 500.perlbench_r (Perl interpreter): execution time and SPECrate versus optimization level.

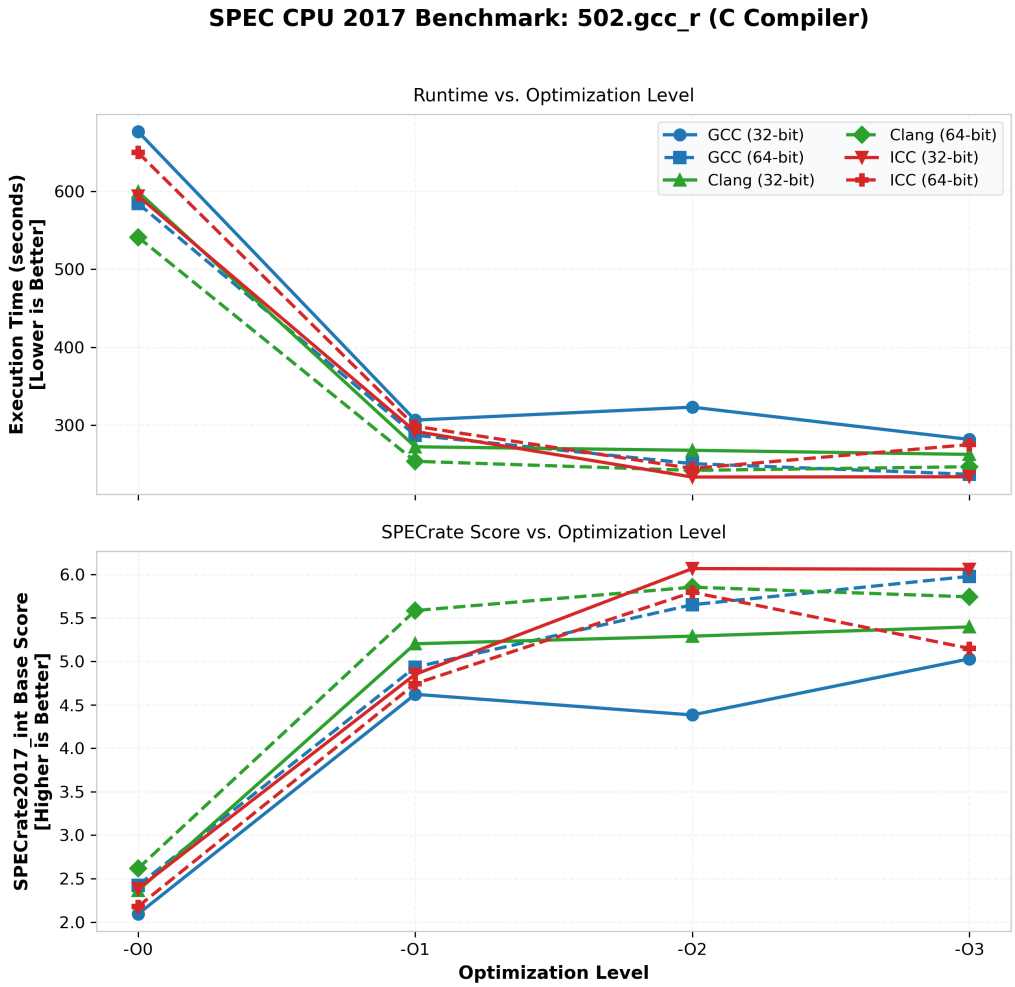

Figure 4: 502.gcc_r (C compiler): execution time and SPECrate versus optimization level.

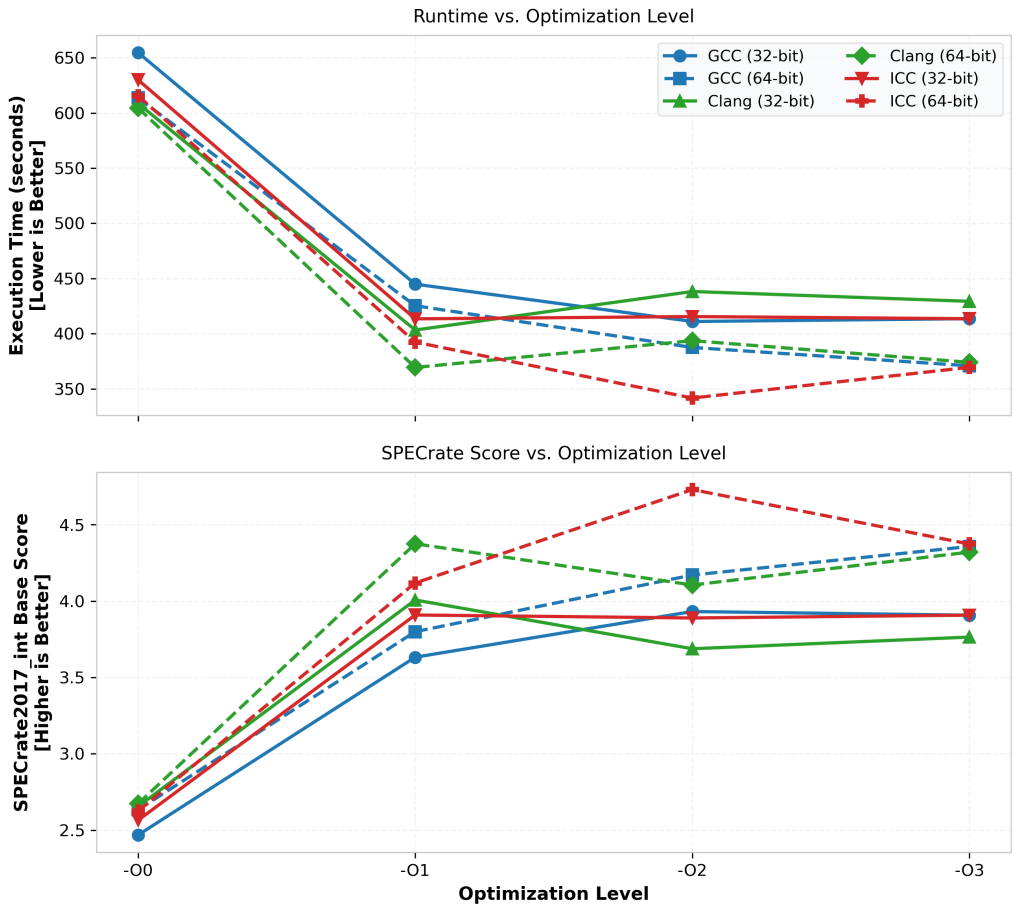

Figure 5: 505.mcf_r (combinatorial optimization / network flow): execution time and SPECrate versus optimization level.

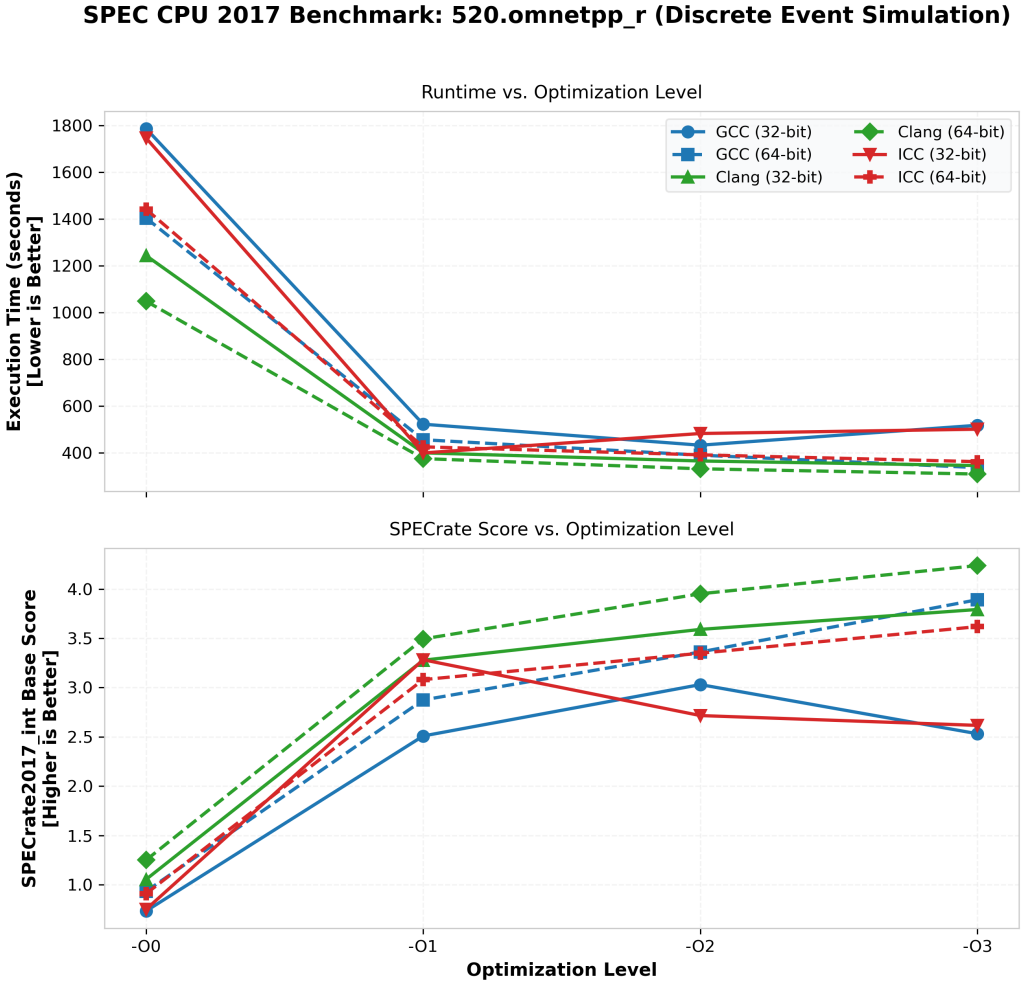

Figure 6: 520.omnetpp_r (discrete-event simulation): execution time and SPECrate versus optimization level.

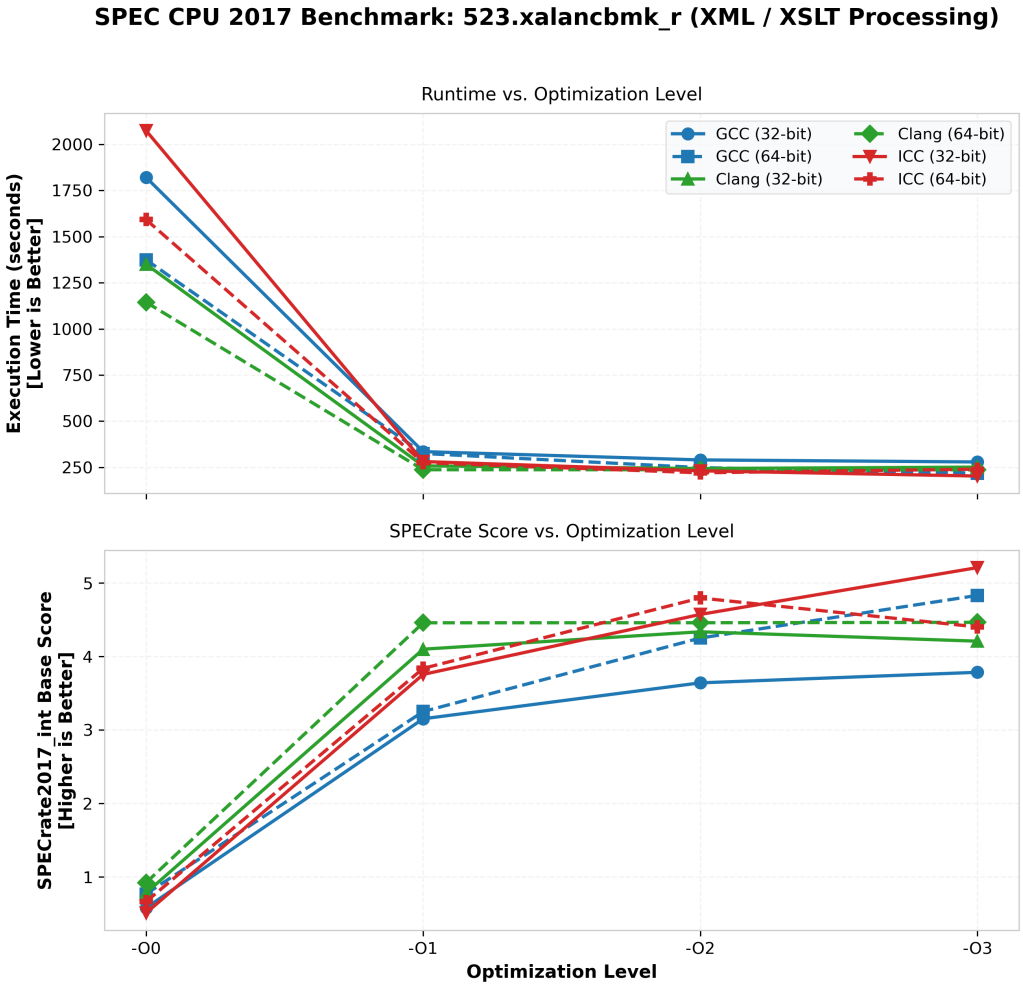

Figure 7: 523.xalancbmk_r (XML/XSLT processing): execution time and SPECrate versus optimization level.

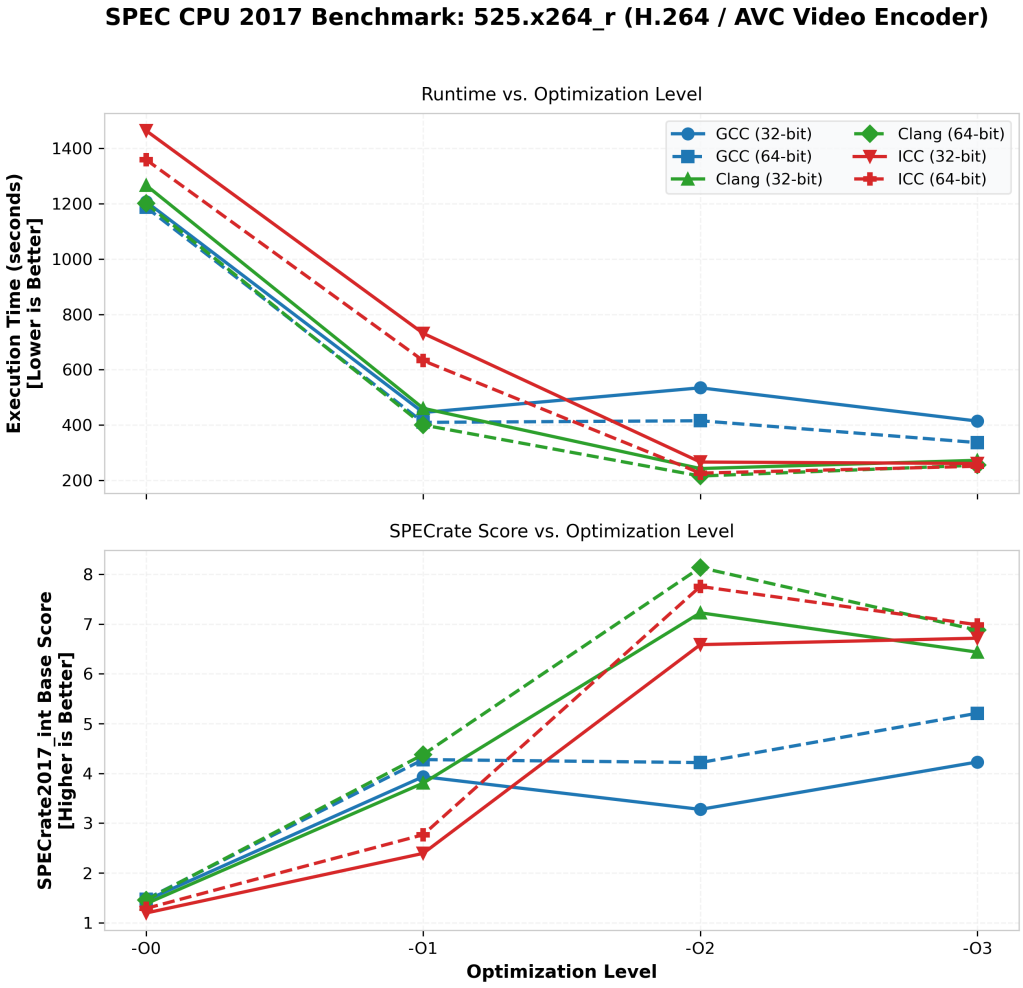

Figure 8: 525.x264_r (H.264/AVC video encoder): execution time and SPECrate versus optimization level.

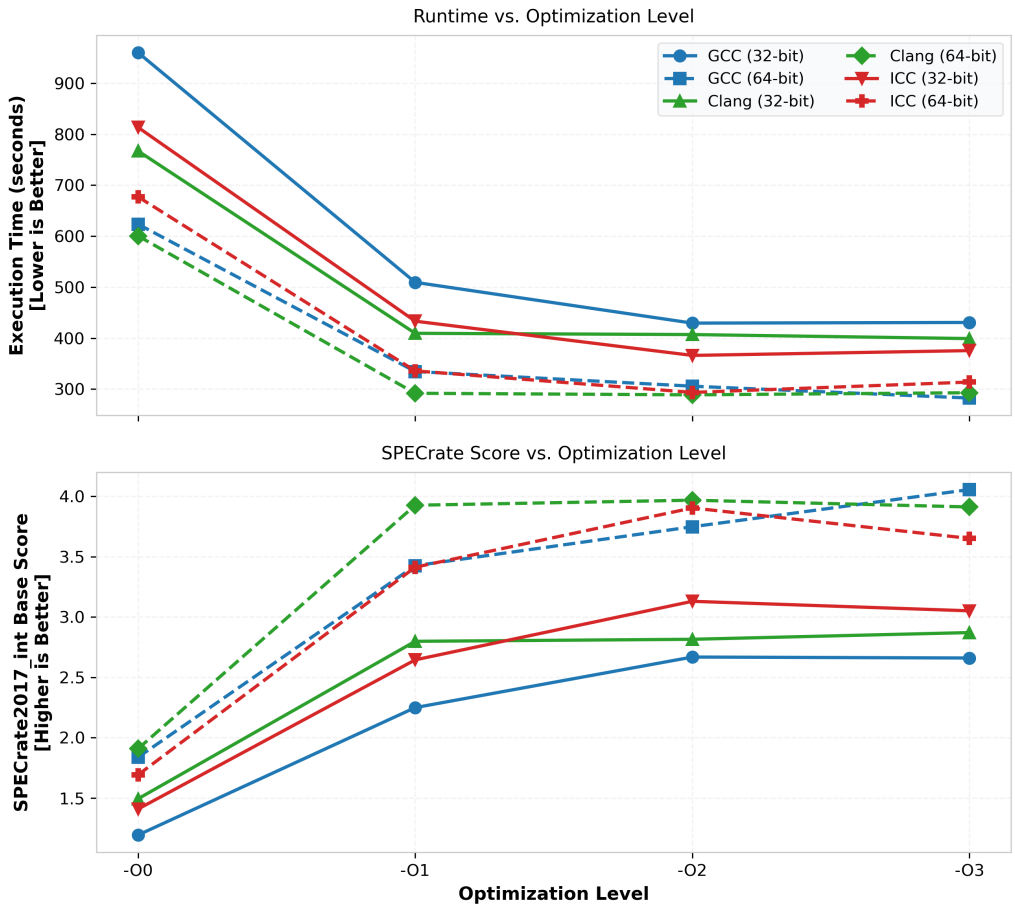

Figure 9: 531.deepsjeng_r (chess search engine): execution time and SPECrate versus optimization level.

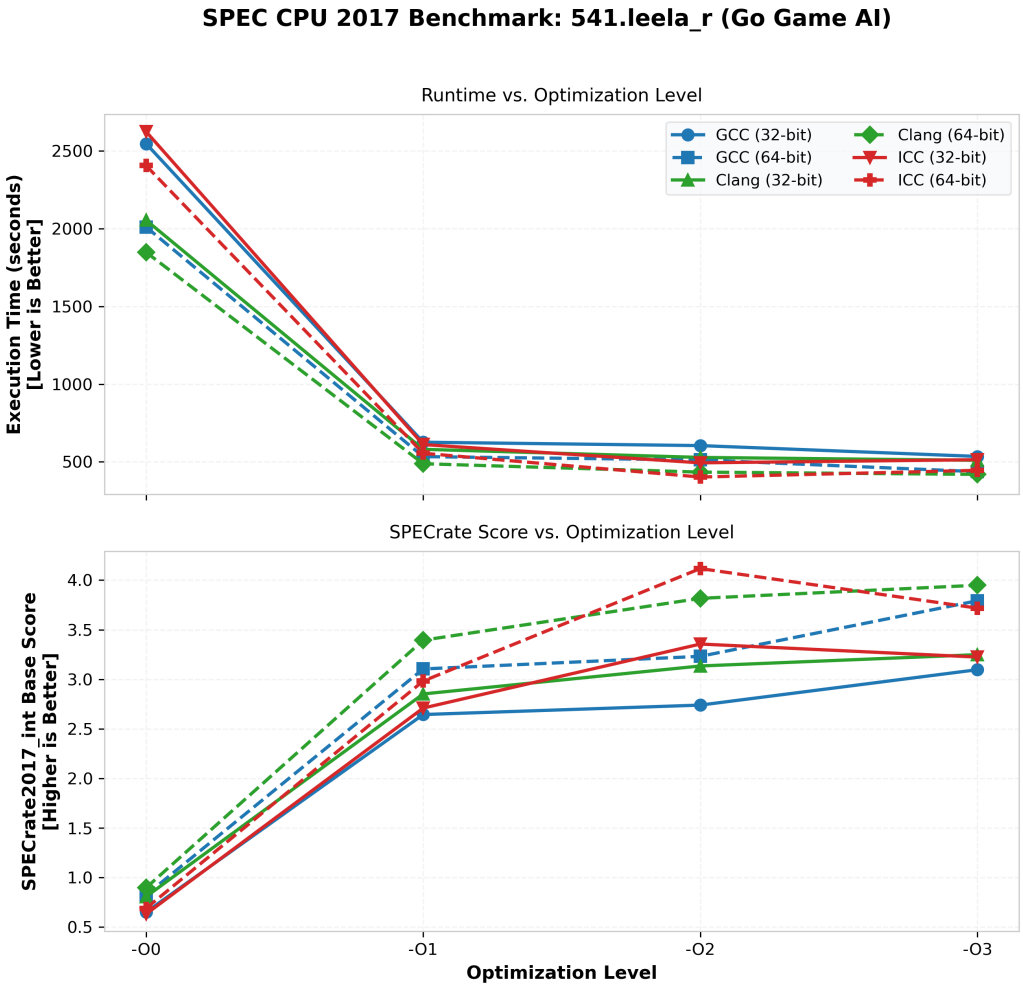

Figure 10: 541.leela_r (Go game AI): execution time and SPECrate versus optimization level.

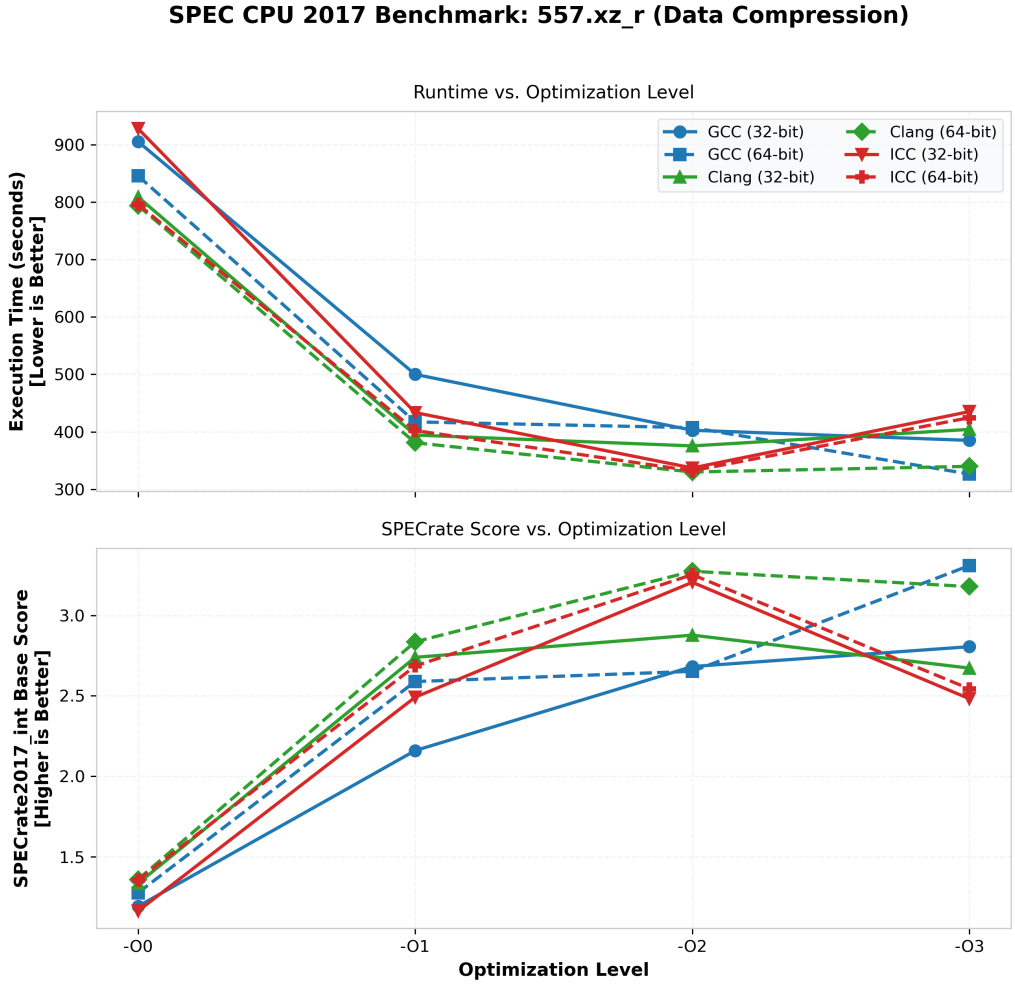

Figure 11: 557.xz_r (data compression): execution time and SPECrate versus optimization level.